

1. GitHub Repository Link

Your repository name must follow this format: IT342-Lastname-AppName

Backend Project Naming Convention (Spring Boot)

The backend project must follow the Maven naming convention below.

Group ID format: edu.cit.lastname

Examples:

- edu.cit.delacruz
- edu.cit.santos
- edu.cit.garcia

Artifact ID format: appname

Examples:

- campusclinic
- busticketing
- studenttracker

Example Maven Configuration

Group: edu.cit.delacruz

Artifact: campusclinic

This will generate a base package: edu.cit.delacruz.campusclinic

The backend must follow the required technology stack below.

- Framework: Spring Boot
- Version: Spring Boot 3.5.x
- Build Tool: Maven
- Architecture: REST API

2. Final Commit for Phase 1

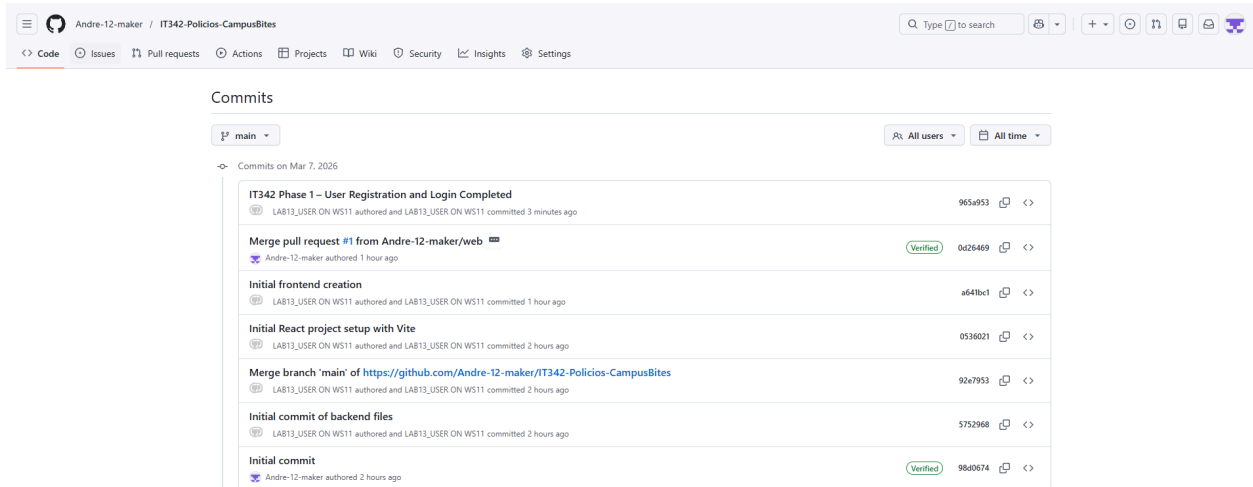
Before the deadline, create a final commit in your repository.

Use the commit message:

IT342 Phase 1 – User Registration and Login Completed

Include the commit link or commit hash in your PDF.

965a95312a1bc84821813b348f61cf3786cd0f91



The screenshot shows the GitHub interface for the repository 'IT342-Policios-CampusBites' by user 'Andre-12-maker'. The 'Commits' tab is selected, showing a list of commits on the 'main' branch. The commits are ordered from most recent to oldest. The most recent commit is 'IT342 Phase 1 – User Registration and Login Completed' with hash 965a953, authored by LAB13_USER ON WS11 3 minutes ago. Other recent commits include a merge pull request, 'Initial frontend creation', 'Initial React project setup with Vite', another merge branch, 'Initial commit of backend files', and an 'Initial commit'.

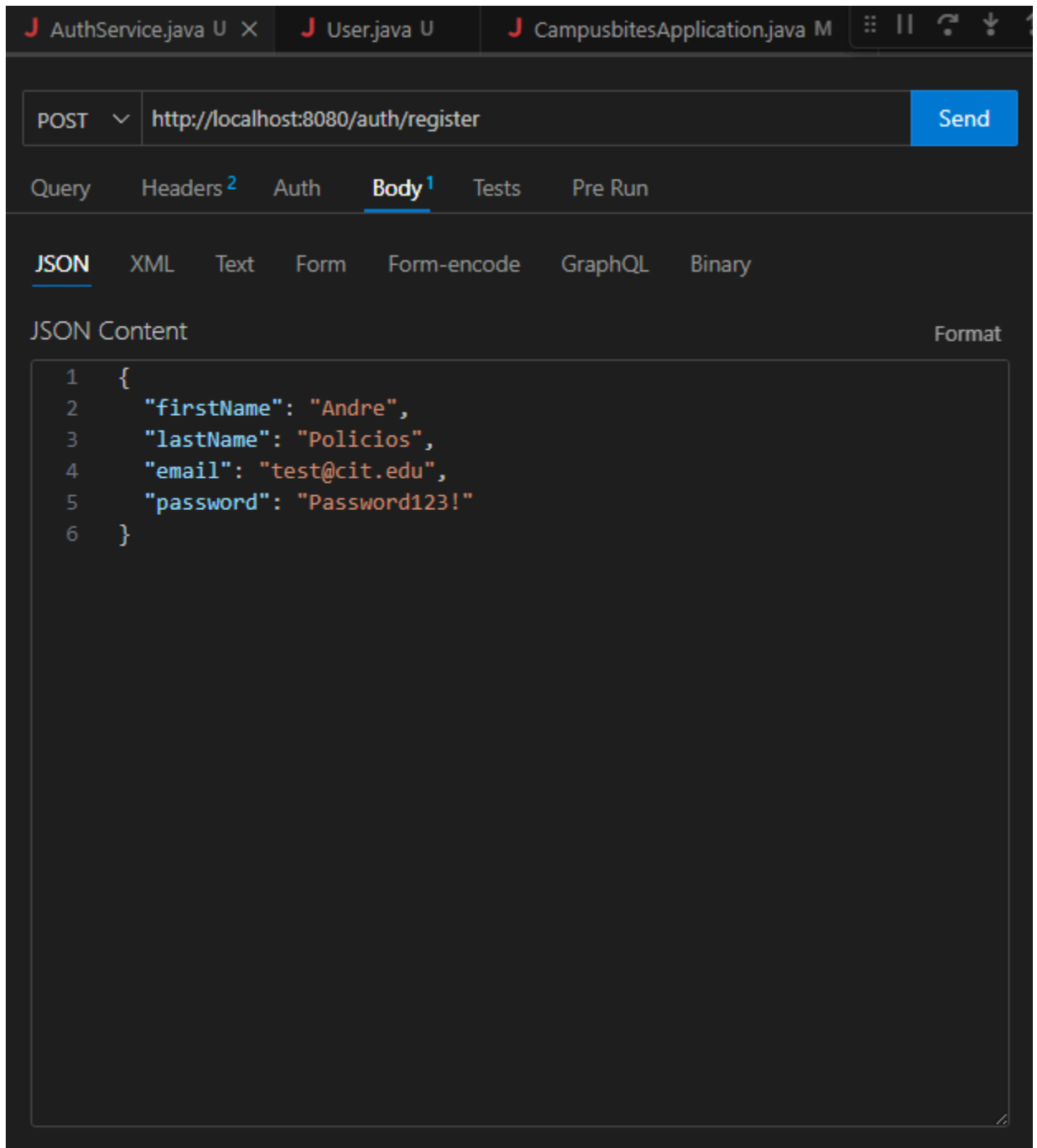
Commit Message	Hash	Author	Time
IT342 Phase 1 – User Registration and Login Completed	965a953	LAB13_USER ON WS11	3 minutes ago
Merge pull request #1 from Andre-12-maker/web	0a26469	Andre-12-maker	1 hour ago
Initial frontend creation	a641bc1	LAB13_USER ON WS11	1 hour ago
Initial React project setup with Vite	0536021	LAB13_USER ON WS11	2 hours ago
Merge branch 'main' of https://github.com/Andre-12-maker/IT342-Policios-CampusBites	92e7953	LAB13_USER ON WS11	2 hours ago
Initial commit of backend files	5752968	LAB13_USER ON WS11	2 hours ago
Initial commit	98d0674	Andre-12-maker	2 hours ago

3. Screenshots of Implementation

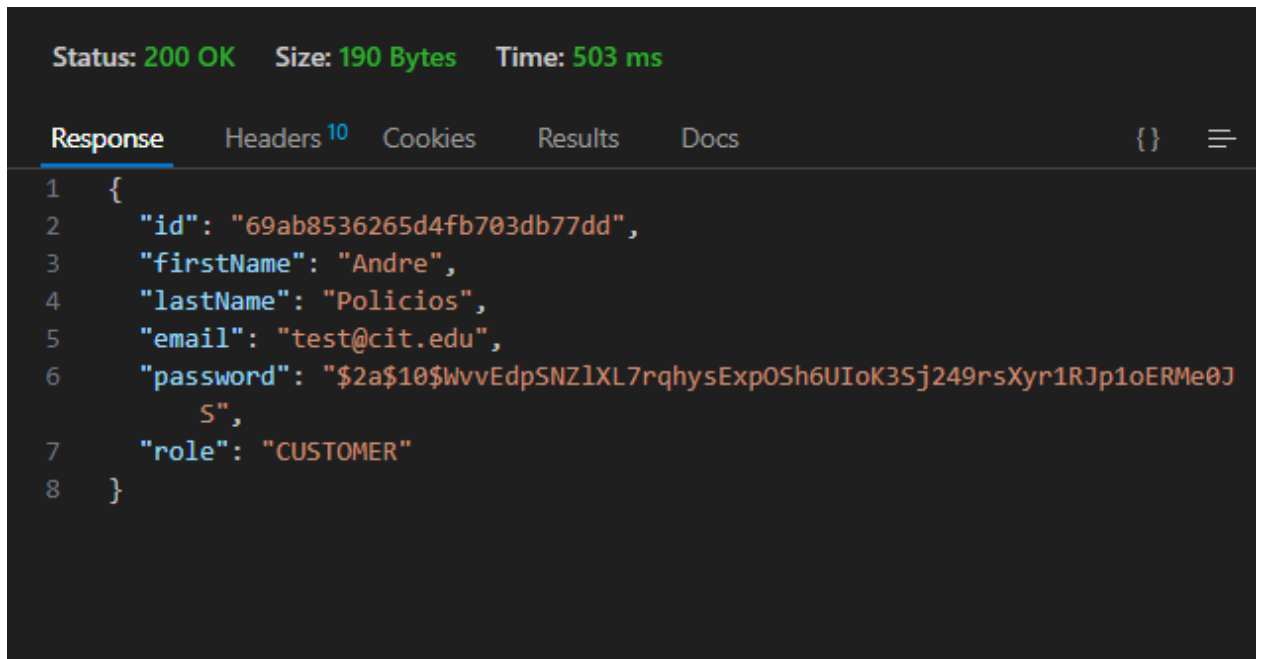
Include screenshots showing the system working.

Required screenshots:

1. Registration page



2. Successful user registration



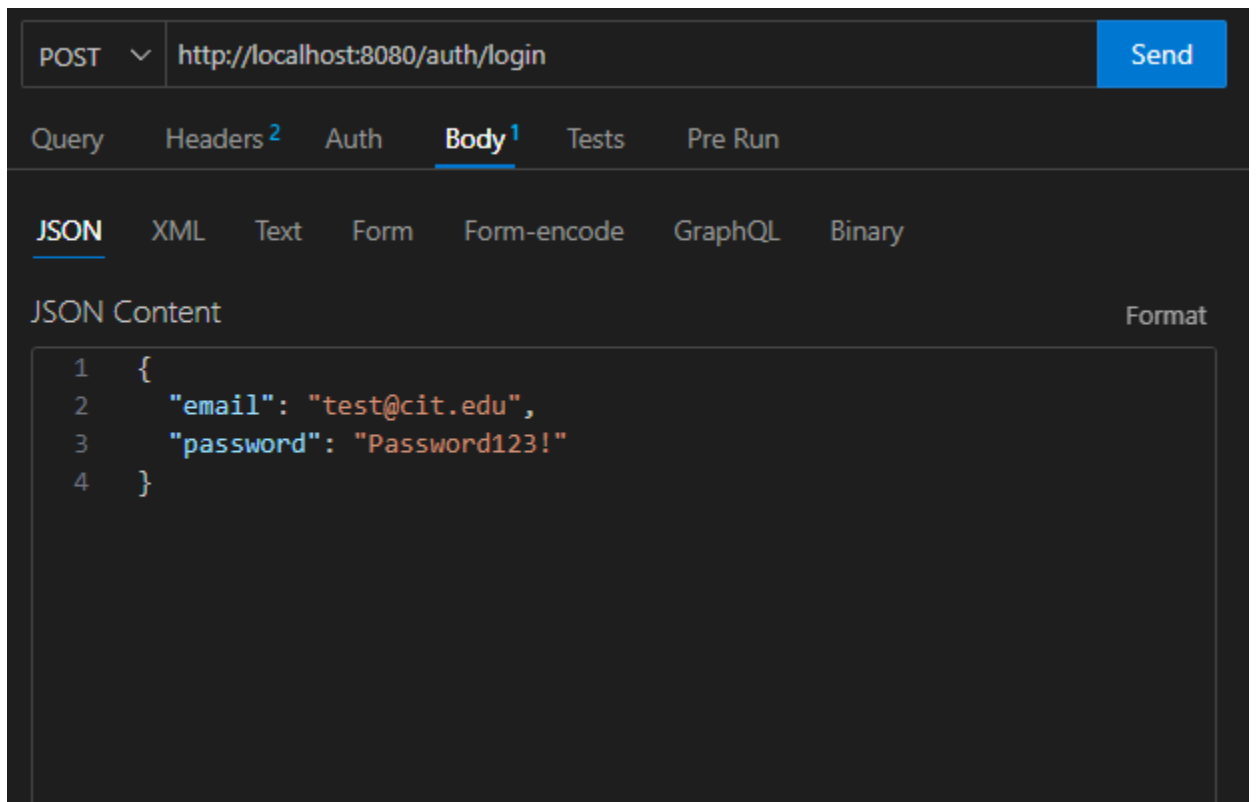
A screenshot of a REST client interface showing the response of a successful user registration. The status is 200 OK, size is 190 Bytes, and time is 503 ms. The response is a JSON object with the following fields: id, firstName, lastName, email, password, and role.

```
Status: 200 OK   Size: 190 Bytes   Time: 503 ms

Response  Headers 10  Cookies  Results  Docs  {}  ≡

1  {
2    "id": "69ab8536265d4fb703db77dd",
3    "firstName": "Andre",
4    "lastName": "Policios",
5    "email": "test@cit.edu",
6    "password": "$2a$10$WvvEdpSNZlXL7rqhysExp0Sh6UIoK3Sj249rsXyr1RJp1oERMe0J
7    "role": "CUSTOMER"
8  }
```

3. Login page



A screenshot of a REST client interface showing a login request. The method is POST and the URL is http://localhost:8080/auth/login. The request body is a JSON object with the following fields: email and password.

```
POST  http://localhost:8080/auth/login  Send

Query  Headers 2  Auth  Body 1  Tests  Pre Run

JSON  XML  Text  Form  Form-encode  GraphQL  Binary

JSON Content  Format

1  {
2    "email": "test@cit.edu",
3    "password": "Password123!"
4  }
```

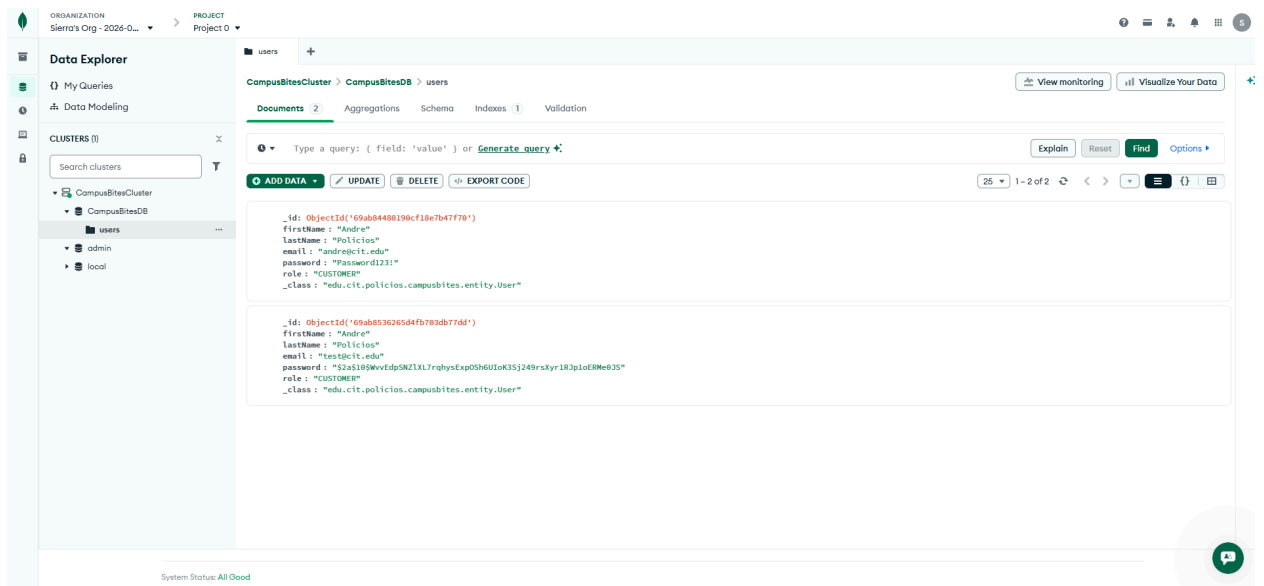
4. Successful login

```
Status: 200 OK   Size: 190 Bytes   Time: 477 ms

Response  Headers 10  Cookies  Results  Docs  {}  ≡

1  {
2    "id": "69ab8536265d4fb703db77dd",
3    "firstName": "Andre",
4    "lastName": "Policios",
5    "email": "test@cit.edu",
6    "password": "$2a$10$WvvEdpSNZlXL7rqhysExp0Sh6UIoK35j249rsXyr1RJp1oERMe0J",
7    "role": "CUSTOMER"
8  }
```

5. Database record showing the registered user



4. Short Implementation Summary (1 Page)

User Registration

- **Registration Fields:** The system captures `firstName`, `lastName`, `email`, and `password` from the client via a JSON payload.
- **Validation Process:** The `AuthService` performs a check against the `UserRepository` to ensure the provided email does not already exist in the database.
- **Duplicate Prevention:** If an email is found, the system throws a runtime exception, preventing the creation of duplicate accounts.
- **Security:** Passwords are never stored in plain text. They are encrypted using the **BCrypt hashing algorithm** before being saved to MongoDB, fulfilling the security requirements of the SDD.

User Login

- **Login Credentials:** Users authenticate by providing their registered `email` and `password`.
- **System Verification:** The backend retrieves the user record by email and utilizes `passwordEncoder.matches()` to verify the raw input against the stored BCrypt hash.
- **Successful Login:** Upon successful verification, the server returns a **200 OK** status along with the user's details and their assigned role.

Database Table

- **Table/Collection:** `users` (Hosted on MongoDB Atlas).
- **Columns/Fields:** `id`, `firstName`, `lastName`, `email`, `password` (hashed), and `role`.

API Endpoints

- **POST `/auth/register`:** Handles new user creation and password hashing.
- **POST `/auth/login`:** Handles user authentication and credential verification.